DATABYT · ENGINEERING CURRICULUM

ADK Mastery

Learn Google's Agent Development Kit from baby steps to master — in Python — then ship a real support agent into DataByt production. Agents, multi-agents, sub-agents, pipelines, the visual UI, and the Vertex ecosystem, explained as simply as possible.

> Brain + Instructions + Tools — that's an agent. Everything else is plumbing.

Python ADK · Gemini Flash · adk web UI · Cloud Run · pgvector · Vertex AI | v1.0 · June 2026

The 30-Second Mental Model

An **agent** is just three things glued together:

```
┌──────────────────────────────────┐
│  AGENT                           │
│    1. A BRAIN      (the LLM — Gemini) │
│    2. INSTRUCTIONS (what its job is)  │
│    3. TOOLS        (things it can DO) │
└──────────────────────────────────┘
```

Everything else in ADK — sessions, memory, multi-agent, callbacks — is plumbing around these three. Hold this picture the whole way through.

Analogy: an agent is a new employee. The **brain** is their intelligence (Gemini), the **instructions** are their job description, the **tools** are the systems they're allowed to use. A **multi-agent system** is a team with a manager who routes work to the right specialist.

PART A

Learn ADK in Python — Baby → Master

We learn in Python because that's where ADK is most mature, where the visual UI (`adk web`) lives, and where the Google ecosystem (Vertex) plugs in cleanly. Understand it here, and moving to DataByt is easy.

Lesson 0 Setup (15 minutes, once)

```
# clean folder + virtual env
mkdir adk-learning && cd adk-learning
python -m venv .venv
.venv\Scripts\activate           # Windows  (Mac: source .venv/bin/activate)
pip install google-adk
```

Get a key at **aistudio.google.com/apikey**. ADK wants each agent in its own folder with two files:

```
greeting_agent/
├── __init__.py    # one line:  from . import agent
├── agent.py       # your agent
└── .env           # GOOGLE_API_KEY=...  &  GOOGLE_GENAI_USE_VERTEXAI=FALSE
```

Lesson 1 Your First Agent

```
from google.adk.agents import Agent

root_agent = Agent(
    name="greeting_agent",
    model="gemini-2.0-flash",
    description="A friendly agent that greets people.",
    instruction="""
    You are a warm, friendly assistant.
    Greet the user by name if they give it.
    Keep replies to one or two sentences.
    """,
)
```

- `root_agent` — ADK looks for this exact variable name. Always name your top agent this.
- `gemini-2.0-flash` — the cheap, fast model. Use it for everything until you have a reason not to.
- `instruction` — the job description. **90% of agent quality is here.**

> **The magic moment — the visual UI.** From the parent folder run `adk web` , open **localhost:8000**: a dropdown to pick your agent, a chat box, an **Events** panel (every step it took), a **State** panel (its memory), a **Trace** view. You build every agent here first and watch what it does. The UI turns the invisible — an LLM thinking — into something you can see.

Lesson 2 Tools — giving the agent hands

An agent with no tools can only *talk*. A tool lets it *do*. In Python a tool is a normal function with type hints and a docstring — ADK reads the docstring to know when to use it.

```python
def get_invoice_count(status: str) -> dict:
    """Returns how many invoices have a given status.
    Args:
        status: One of 'open', 'overdue', or 'paid'.
    """
    data = {"open": 12, "overdue": 5, "paid": 240}
    return {"status": "success", "count": data.get(status, 0)}

root_agent = Agent(
    name="ar_helper", model="gemini-2.0-flash",
    instruction="Use get_invoice_count for counts. Never guess numbers.",
    tools=[get_invoice_count],
)
```

Ask *"How many invoices are overdue?"* and watch Events: the agent calls `get_invoice_count("overdue")`, gets `5`, writes a sentence with it.

> **Golden rules of tools:** the docstring is the instruction manual — write it like you're training a new hire. One job per tool. Always return a dict with a `status` key. Type-hint every argument — that's how ADK tells Gemini what to pass.

Lesson 3 Sessions & State — memory in a conversation

A **session** is one conversation. **State** is the agent's scratchpad — a notebook it writes in between turns.

```python
from google.adk.tools.tool_context import ToolContext

def remember_company(name: str, tool_context: ToolContext) -> dict:
    """Saves the user's company name for later."""
    tool_context.state["company_name"] = name   # write to the notebook
    return {"status": "success", "saved": name}
```

Any tool that takes `tool_context: ToolContext` can read/write `state`. In `adk web`, the State panel shows `company_name` appear — the agent's memory, made visible.

The `output_key` shortcut auto-saves an agent's whole reply to state: `Agent(..., output_key="last_summary")`. This is the conveyor belt that moves data between steps in a pipeline.

Prefix	Lives for	Example
(none)	this session	`state["current_ticket"]`
`user:`	all sessions for this user	`state["user:tone"]`
`app:`	everyone, app-wide	`state["app:hours"]`
`temp:`	this turn only	`state["temp:raw"]`

Lesson 4 Multi-Agent — a Manager & Specialists

Instead of one agent doing everything badly, build **specialists** and a **manager** that routes to them.

```
billing_agent = Agent(name="billing_agent", model="gemini-2.0-flash",
    description="Handles billing, pricing, refunds, invoices.",
    instruction="You are a billing specialist.")

tech_agent = Agent(name="tech_agent", model="gemini-2.0-flash",
    description="Handles QuickBooks, Xero, syncing, OAuth.",
    instruction="You are a technical support specialist.")

root_agent = Agent(name="support_manager", model="gemini-2.0-flash",
    instruction="""Route the ticket:
      billing/pricing/refunds/invoices -> billing_agent
      QuickBooks/Xero/sync/connection  -> tech_agent
      Do not answer specialist questions yourself. Route them.""",
    sub_agents=[billing_agent, tech_agent])   # the team
```

The key line is `sub_agents=[...]`. The manager can **transfer** control to whichever specialist fits. Ask *"Why won't my QuickBooks sync?"* and watch Events: the manager transfers to `tech_agent`, which answers.

> **This is the single most important pattern in ADK.** A manager + specialists. It's how you handle many request types without one giant unmaintainable instruction. Specialists can have their own specialists — that's a tree. Don't over-nest early; two levels is plenty.

Lesson 5 Workflow Agents — controlled pipelines

Manager-routing is *LLM-decided*. Sometimes you want a **guaranteed order**. Three workflow agents do that:

```
from google.adk.agents import SequentialAgent

classifier = Agent(..., instruction="Classify: billing/technical/how-to.",
                   output_key="category")
responder  = Agent(..., instruction="Category is {category}. Write a reply.")

pipeline = SequentialAgent(name="support_pipeline",
                           sub_agents=[classifier, responder])  # in order
root_agent = pipeline
```

`{category}` pulls what the classifier saved via `output_key` — **state is the conveyor belt**.

You want…	Use
The LLM to decide who handles it	`sub_agents` on a normal Agent
A guaranteed step-by-step order	`SequentialAgent`
Several independent things at once (faster)	`ParallelAgent`
Repeat-until-good	`LoopAgent` (`max_iterations`)

Lesson 6 Agent-as-a-Tool — clean delegation

Sometimes you don't want to *transfer* — you want one agent to *call* another and get an answer back, like a function.

```
from google.adk.tools import agent_tool

translator = Agent(name="translator", model="gemini-2.0-flash",
    instruction="Translate the text to formal business English.")

main_agent = Agent(name="email_writer", model="gemini-2.0-flash",
    instruction="Write dunning emails. Use the translator tool to formalise.",
    tools=[agent_tool.AgentTool(agent=translator)])
```

Transfer vs Agent-as-Tool: transfer hands over the whole conversation (route to billing); Agent-as-Tool keeps the main agent in charge — it calls the helper, gets a result, continues (go translate this, then come back).

Lesson 7 Callbacks — guardrails & control

Callbacks are hooks that run *around* the agent's steps — security checkpoints for safety, logging, cost.

```
def block_bad(callback_context, llm_request):
    """Runs BEFORE the LLM. Returning a response blocks the call."""
    text = llm_request.contents[-1].parts[0].text.lower()
    if "delete everything" in text:
        return LlmResponse(...)   # short-circuit, never calls LLM
    return None               # None = proceed normally

agent = Agent(..., before_model_callback=block_bad)
```

Callback	Use it for
`before_model_callback`	guardrails — block/modify input before the LLM
`after_model_callback`	filter or redact output
`before_tool_callback`	validate args, block a dangerous tool call
`after_tool_callback`	reshape a tool's result
`before/after_agent_callback`	skip the agent / final logging

For DataByt the big two: `before_model` (don't promise refunds it can't give) and `before_tool` (don't resend an email without org permission).

Lesson 8 Memory & RAG — knowledge across conversations

State is memory *within* one conversation. **Memory / RAG** is knowledge *across* conversations and from your documents. This is how the agent learns DataByt's whole knowledge base.

```
Your docs -> chopped into chunks -> turned into "embeddings"
(numbers that capture meaning) -> stored in a vector database

User asks -> question -> embedding -> find most similar chunks ->
hand them to the agent -> it answers using YOUR docs
```

> **The big idea:** the agent's knowledge IS your documentation. Every guide doc you write becomes something it can retrieve and answer from. Your `DataByt-Guide/` folder is the agent's brain food. For DataByt we store embeddings in **pgvector inside Supabase** — already there, no new vendor, no cost.

Lesson 9 Models & the Vertex Switch

Everything so far ran on the simple **Gemini API** (a `GOOGLE_API_KEY`). **Vertex AI** is the same Gemini, inside Google Cloud, with enterprise features. You flip to it by changing `.env` — no code change:

```
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
```

You'd do this at production time to run on your Cloud account and credits. ADK can also use non-Gemini models via `LiteLlm` — but for DataByt, stick with Gemini Flash (cheapest, credits cover it). You're not locked in.

Lesson 10 Where You Are Now

> **If you did Lessons 0–9, you can already:** build a single agent · give it tools · give it memory · build a manager + specialists · build Sequential/Parallel/Loop pipelines · have agents call each other · add guardrails · understand RAG and Vertex. **That's the whole toolbox.** A "complex agent" is just these pieces combined well — there is no secret Lesson 50.

PART B

Build the DataByt Support Agent (Python)

Assemble the pieces into the real thing — a support agent that resolves 70% of tickets. It's just Lesson 4 (manager + specialists) + Lesson 2 (tools) + Lesson 8 (RAG). Nothing new.

```
support_manager  (routes the ticket)
├── knowledge_agent  -> answers "how do I..." via RAG over DataByt-Guide
├── account_agent    -> checks real data
│       tools: get_invoice_status, get_integration_status, resend_email
└── escalation_agent -> hands hard tickets to Kuberan with full context
        tools: escalate_to_human
```

The manager (the heart of it)

```
root_agent = Agent(name="support_manager", model="gemini-2.0-flash",
  instruction="""Route each ticket:
    'How do I...', product questions      -> knowledge_agent
    status of invoice / sync / connection -> account_agent
    refunds, disputes, bugs, anger, unclear -> escalation_agent
    Always route. Never invent an answer yourself.""",
  sub_agents=[knowledge_agent, account_agent, escalation_agent])
```

An action tool the account agent uses:

```
def get_invoice_status(invoice_number: str, org_id: str) -> dict:
    """Looks up one invoice's current status."""
    r = httpx.get(f"{DATABYT_API}/invoices/lookup",
                  params={"invoice_number": invoice_number, "org_id": org_id})
    return {"status": "success", "invoice": r.json()}
```

Test it in the UI. Run `adk web` and throw real tickets: *"How do I connect QuickBooks?"* (-> knowledge), *"Is INV-1042 paid?"* (-> account, calls the tool), *"I want a refund"* (-> escalation). The Events panel is your test harness — wrong routing? fix the manager's instruction and retry. No deploy needed to test.

PART C

Get It Into DataByt Production (the easy way)

The honest truth, simply: ADK is most reliable in Python. DataByt is TypeScript. The cleanest bridge is — run the Python agent as a tiny service, and have DataByt call it over HTTPS. This is not scary. Google gives you a one-command deploy.

```
Customer -> DataByt (Next.js / Vercel) -> /api/support route
                                              |  fetch() HTTPS
                                              v
                                   ADK agent (Python) on Cloud Run
                                   (scales to zero = ₹0 when idle)
                                              |
                             calls DataByt's API (invoice status) + Gemini
```

Step 1 — make the agent an API

```
adk api_server      # serves your agent with a /run endpoint at :8000
```

Step 2 — deploy to Cloud Run (one command)

```
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
adk deploy cloud_run --project YOUR_PROJECT_ID \
  --region us-central1 ./support_manager
# ~3 min later -> https://support-manager-xxxxx-uc.a.run.app
```

That's your agent, live, on your Cloud credits, scaling to zero. **Even easier:** `adk deploy agent_engine` hosts it fully-managed by Google *with a built-in UI + tracing dashboard* (Part D).

Step 3 — wire DataByt to call it (one route)

```
// src/app/api/support/route.ts
export async function POST(req: NextRequest) {
  const { message, orgId, ticketId } = await req.json();
  const r = await fetch(`${AGENT_URL}/run`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      app_name: "support_manager", user_id: orgId, session_id: ticketId,
      new_message: { role: "user", parts: [{ text: message }] },
    }),
  });
  return NextResponse.json({ reply: await r.json() });
}
```

Add `SUPPORT_AGENT_URL` to Vercel env vars. **That's the whole integration** — one route, one env var.

Step 4 — push to production

```
adk deploy cloud_run ...      # redeploy the agent when it changes
git push                      # Vercel auto-deploys DataByt
```

No Docker to hand-write, no Kubernetes, no servers to manage.

My recommendation for you, right now: (1) Learn & prototype in Python ADK — the UI makes learning 10× faster. (2) Ship via Cloud Run — genuinely one command, scales to zero. (3) Later, if the second service annoys you, port the proven agent to TypeScript in-process (DataByt already calls Gemini in TS) — by then you'll understand it cold and the port is an afternoon. **Don't optimise for "one language" before you've shipped. Optimise for understanding fast and shipping reliably.**

PART D

The UI & the Ecosystem (Vertex, simply)

The `adk web` dev UI — your daily driver

Run it locally; this is where you live while building. **Chat** (talk like a customer), **Events** (every routing + tool call — this is how you debug), **State** (live memory), **Trace** (timing), **Eval** (run saved test questions).

95% of your ADK time is in `adk web`. When the agent misbehaves, the Events panel tells you exactly where. Master this one screen and you've mastered the workflow.

Vertex AI — only the 3 pieces DataByt needs

Vertex piece	What it does for you	When
Vertex AI Studio	Try prompts / pick models on your Cloud project	While tuning instructions
Agent Engine	Managed agent hosting + built-in trace/monitoring UI	When you deploy for real
Cloud Logging	Every production run, errors, token usage	After launch, watch health

Ignore for now (for big ML teams, not a solo AR SaaS): Vertex Vector Search (use pgvector — free), Model Garden tuning, Pipelines, Feature Store, BigQuery ML, Dataflow.

Agent Engine's UI is your production cockpit: chat with the live agent, see traces of real customer tickets, watch token cost per request, spot failures. It's `adk web` for your live agent, hosted by Google.

PART E

Your Learning Path & Checklist

```
WEEK 1 — Single agents & tools   (Lessons 0–3)
  Build 3 toy agents in adk web. Comfortable with Agent, instruction,
  tools, state. -> an agent that uses a tool and remembers something.

WEEK 2 — Multi-agent & pipelines  (Lessons 4–7)
  Build manager+specialists, add a guardrail. Routing feels natural.
  -> the DataByt support tree, routing correctly in the UI.

WEEK 3 — Knowledge & ship         (Lesson 8–9 + Parts B–C)
  Add RAG over DataByt-Guide, deploy to Cloud Run, wire /api/support.
  -> a customer asks "how do I connect QuickBooks" and the LIVE agent answers.
```

The one rule that makes you a master: never learn for more than two days without building the thing in `adk web`. Reading about agents teaches you nothing. Watching the Events panel show a wrong routing decision, fixing the instruction, and seeing it route right — *that* is the learning. Live in the UI.

Quick glossary

Word	Plain meaning	Word	Plain meaning
Agent	brain + instructions + tools	Transfer	hand the whole convo to a sub-agent
Tool	a function the agent can call	AgentTool	call another agent like a function
Instruction	the agent's job description	SequentialAgent	run steps in fixed order
Session	one conversation	ParallelAgent	run steps at once
State	notebook within a conversation	LoopAgent	repeat until good
Memory	knowledge across convos / docs	Callback	hook before/after a step
Sub-agent	a specialist a manager routes to	RAG	search your docs before answering
Cloud Run	cheap hosting, scales to zero	Agent Engine	Google's managed agent hosting + UI

You started as a baby. Do the three weeks, live in `adk web`, and you finish a master — with a real support agent answering real CFOs in production. That is the whole game.